

PROJETO INTEGRADOR

Análise e Desenvolvimento de Sistemas — 4º Semestre

Documentação de Arquitetura

Modelo C4 — Contexto e Contêineres

Solução Distribuída para Logística e Rastreamento de Entregas

2026/1

1. Introdução e Visão Geral

Este documento descreve a arquitetura do sistema de logística e rastreamento de entregas desenvolvido como Projeto Integrador do curso de Análise e Desenvolvimento de Sistemas (4º Semestre, 2026/1). A documentação segue o modelo C4, abrangendo o Diagrama de Contexto (nível 1) e o Diagrama de Contêineres (nível 2).

O sistema atende ao seguinte problema central: o crescimento do e-commerce exige plataformas capazes de processar pedidos, notificar status em tempo real e gerenciar falhas de forma resiliente — sem bloquear a experiência do usuário durante o processamento assíncrono.

Objetivos do Sistema

- Permitir que clientes realizem pedidos de produtos (camisetas) por meio de interface web.
- Processar pedidos de forma assíncrona utilizando Apache Kafka como broker de mensageria.
- Atualizar o status do pedido em tempo real no navegador via WebSocket, sem polling.
- Garantir desacoplamento entre os serviços de backend por meio de eventos de domínio.
- Orquestrar toda a infraestrutura com Docker Compose, facilitando deploy e reprodutibilidade.

Stack Tecnológica

Camada	Tecnologia / Justificativa
Backend	Java 21 + Spring Boot 3.x — runtime moderno com suporte a Virtual Threads e ecossistema robusto para microserviços.
Mensageria	Apache Kafka — broker distribuído de alta throughput; desacopla produtores e consumidores; permite replay e auditoria de eventos.
Banco de Dados	PostgreSQL — RDBMS confiável para persistência de pedidos, usuários e catálogo.
Cache	Redis — cache de sessão e filas de notificação temporárias.
Frontend	React + Vite — SPA com atualização reativa via SockJS/STOMP para exibição de status em tempo real.
Infraestrutura	Docker Compose — orquestra todos os serviços em containers isolados com rede interna compartilhada.

2. Nível 1 — Diagrama de Contexto (C1)

O Diagrama de Contexto apresenta o sistema como uma caixa preta, identificando todos os atores externos (pessoas e sistemas) que interagem com ele. É o nível mais alto de abstração do modelo C4.



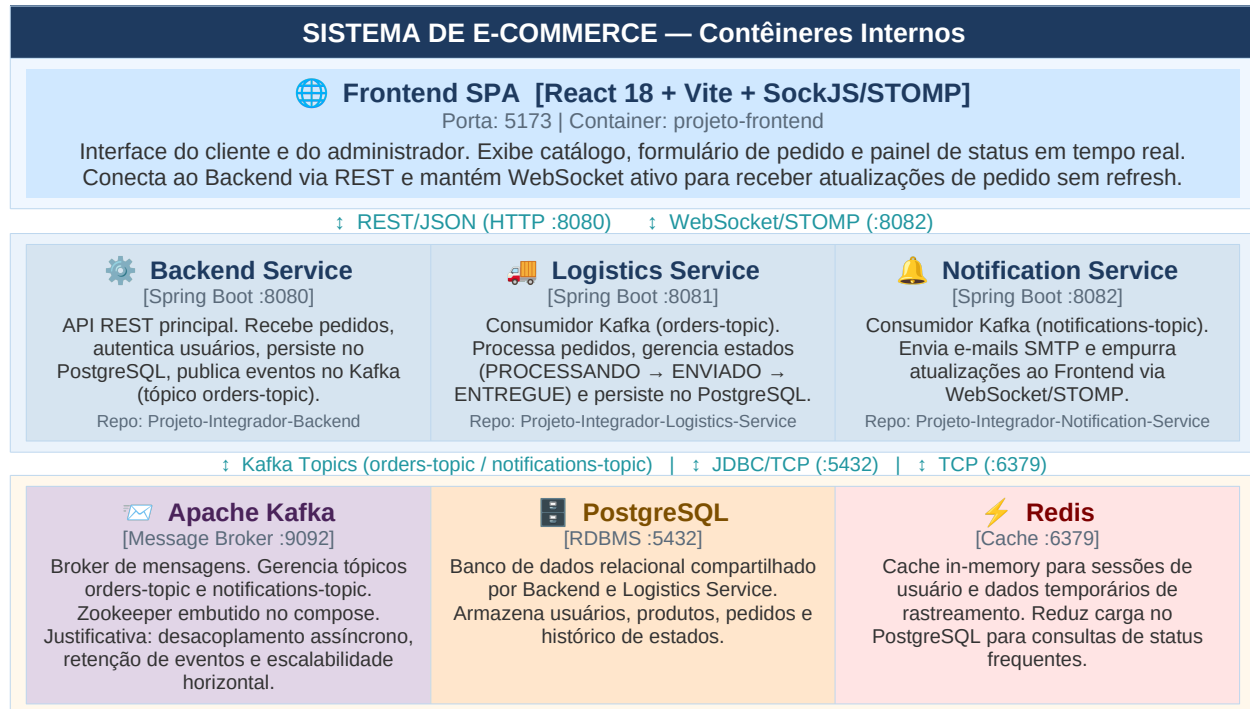
Atores e Sistemas Externos

Elemento	Tipo	Descrição
Cliente	Pessoa	Usuário final que navega pelo catálogo, realiza compras e acompanha o status de entrega em tempo real via interface web.
Administrador	Pessoa	Operador interno que gerencia catálogo de produtos, estoque, pedidos e monitora a saúde do sistema via dashboard.
Serviço de E-mail	Sistema Externo	Provedor SMTP (ex: SendGrid) acionado pelo Notification Service para envio de e-mails transacionais.
Gateway de Pagamento	Sistema Externo	API de terceiro (ex: Stripe) para autorização e captura de pagamentos. Integrado ao Backend Service via REST.

3. Nível 2 — Diagrama de Contêineres (C2)

O Diagrama de Contêineres detalha o interior do sistema, mostrando os contêineres implantáveis (aplicações, bancos de dados, message brokers) e como eles se comunicam. Cada contêiner representa um processo separado que pode ser implantado e escalado de forma independente.

[DIAGRAMA DE CONTÊINERES — C2]



Descrição dos Contêineres

Contêiner	Tecnologia	Porta	Responsabilidade Principal
Frontend SPA	React + Vite	5173	Interface do cliente/admin. Renderiza catálogo, envia pedidos via REST e exibe atualizações de status em tempo real via WebSocket/STOMP.
Backend Service	Spring Boot 3.x	8080	API REST principal. CRUD de produtos, usuários e pedidos. Publica eventos OrderCreatedEvent no tópico orders-topic do Kafka.
Logistics Service	Spring Boot 3.x	8081	Microserviço de pedidos. Consome orders-topic, gerencia a máquina de estados do pedido (PROCESSANDO → ENVIADO → ENTREGUE) e persiste transições.
Notification Service	Spring Boot 3.x	8082	Consome notifications-topic. Envia e-mails transacionais via SMTP e empurra eventos de status ao Frontend conectado via WebSocket/STOMP.
Apache Kafka	Kafka + Zookeeper	9092	Message broker. Gerencia os tópicos orders-topic (pedidos) e notifications-topic (alertas). Garante entrega assíncrona e desacoplamento entre serviços.
PostgreSQL	PostgreSQL 15	5432	Banco de dados relacional. Armazena domínios: usuários, produtos, pedidos e histórico de transições de estado.

Redis	Redis 7	6379	Cache in-memory para sessões de usuário autenticado e dados de rastreamento frequentemente consultados.
--------------	---------	------	---

4. Fluxo de Eventos — Ciclo de Vida de um Pedido

O fluxo abaixo descreve o caminho completo de um pedido, desde a ação do cliente até a atualização em tempo real na interface. Este é o fluxo central que demonstra o uso da mensageria assíncrona como diferencial arquitetural.

#	Ator / Componente	Ação
1	Cliente (Browser)	Preenche formulário de pedido e clica em Confirmar. O Frontend envia POST /orders para o Backend Service via REST/JSON.
2	Backend Service	Valida a requisição, persiste o pedido no PostgreSQL com status PENDENTE e publica o evento OrderCreatedEvent no tópico Kafka orders-topic.
3	Apache Kafka	Recebe a mensagem no tópico orders-topic e a mantém disponível para consumidores subscritos. A resposta ao cliente (HTTP 202 Accepted) já foi enviada — sem bloqueio.
4	Logistics Service	Consome a mensagem de orders-topic, processa a lógica de logística e atualiza o estado do pedido para PROCESSANDO. Publica evento de status no tópico notifications-topic.
5	Notification Service	Consome notifications-topic. Envia e-mail de confirmação ao cliente (SMTP) e emite atualização via WebSocket STOMP no destino /topic/orders/{id}.
6	Frontend (WebSocket)	O browser, com conexão WebSocket ativa, recebe o evento e atualiza o badge de status do pedido em tempo real sem recarregamento de página.
7	Logistics Service	Simula progressão: após intervalo, publica novos eventos (ENVIADO, ENTREGUE) que repetem os passos 5 e 6 até o estado final.

Tópicos Kafka

Tópico	Partições	Produtor	Consumidor(es)
orders-topic	1 (dev) / 3 (prod)	Backend Service	Logistics Service
notifications-topic	1 (dev) / 3 (prod)	Logistics Service	Notification Service

5. Estrutura de Repositórios

O projeto é organizado em múltiplos repositórios sob uma organização GitHub do grupo, seguindo o princípio de separação de responsabilidades por serviço.

Repositório	Tipo	Conteúdo Principal
Projeto-Integrador-Infra	Infraestrutura	docker-compose.yml master, arquivos .env, configurações de rede e volumes Docker.
Projeto-Integrador-Backend	Microserviço	API REST principal (Spring Boot). CRUD, autenticação, produtor Kafka.
Projeto-Integrador-Logistics-Service	Microserviço	Consumidor Kafka orders-topic. Máquina de estados do pedido. Produtor notifications-topic.
Projeto-Integrador-Notification-Service	Microserviço	Consumidor Kafka notifications-topic. WebSocket STOMP. SMTP.
Projeto-Integrador-Frontend	Frontend	SPA React + Vite. Interface do cliente e admin. SockJS/STOMP para tempo real.

6. Justificativa Arquitetural — Kafka vs. REST Síncrono

Um dos requisitos acadêmicos mais relevantes deste projeto é o uso obrigatório de mensageria. A tabela abaixo justifica a escolha do Apache Kafka em detrimento de chamadas REST diretas entre serviços.

Critério	REST Síncrono	Apache Kafka (escolha)
Acoplamento	Alto — o produtor precisa conhecer o endpoint do consumidor.	Baixo — produtor publica no tópico; consumidores são independentes.
Disponibilidade	Se o consumidor cair, o produtor recebe erro imediato.	Mensagens ficam retidas no broker até o consumidor subir novamente.
Escalabilidade	Cada nova integração exige novo endpoint e versionamento.	Novos consumidores se inscrevem no tópico sem alterar o produtor.
Rastreabilidade	Sem histórico de chamadas nativo.	Eventos são persistidos e podem ser reprocessados (replay).
Desempenho	Bloqueante — o cliente espera cada etapa concluir.	Não-bloqueante — Backend retorna 202 imediatamente após publicar.
UX (tempo real)	Requer polling periódico do frontend para verificar mudanças.	WebSocket empurra atualizações ao browser sem polling.

7. Decisões Arquiteturais (ADRs Resumidas)

ADR-01 — Arquitetura de Microserviços

Contexto: O sistema precisa evoluir de forma independente em cada domínio (catálogo, pedidos, notificações).

Decisão: Adotar microserviços em vez de monolito.

Consequências:

- (+) Cada serviço pode ser deployado, escalado e versionado de forma independente.
- (+) Falhas em um serviço não derrubam os demais.
- (–) Maior complexidade operacional — mitigada com Docker Compose.

ADR-02 — Comunicação Assíncrona via Kafka

Contexto: O processamento de pedidos não deve bloquear a resposta ao cliente.

Decisão: Usar Apache Kafka como broker de eventos entre Backend, Logistics e Notification Service.

Consequências:

- (+) Desacoplamento temporal — serviços não precisam estar disponíveis simultaneamente.
- (+) Rastreabilidade — eventos persistidos e reprocessáveis.
- (–) Overhead de configuração — aceito dado o contexto educacional da disciplina de Mensageria.

ADR-03 — WebSocket/STOMP para Notificações em Tempo Real

Contexto: O frontend precisa exibir mudanças de status sem polling.

Decisão: Usar Spring WebSocket + STOMP no Notification Service; SockJS no React.

Consequências:

- (+) UX fluida — atualizações chegam em < 1s sem recarregar a página.
- (+) Menor tráfego de rede comparado a polling a cada 5s.
- (–) Conexão persistente por usuário — aceitável para escala do projeto.

8. Infraestrutura — Docker Compose

Toda a infraestrutura é orquestrada via docker-compose.yml no repositório Projeto-Integrador-Infra. Os serviços compartilham a rede interna ecommerce-network.

Serviço Docker	Porta Host	Porta Container	Observações
zookeeper	2181	2181	Coordenação do cluster Kafka. Embutido no Compose para simplicidade.
kafka	9092	9092	Broker principal. KAFKA_ADVERTISED_LISTENERS aponta para host.docker.internal.
postgres	5432	5432	Volume nomeado para persistência entre restarts do container.
redis	6379	6379	Modo standalone. Sem autenticação em dev.
backend	8080	8080	depends_on: postgres, kafka. Spring Profile: docker.

logistics-service	8081	8081	depends_on: kafka, postgres. Consumer Group: logistics-group.
notification-service	8082	8082	depends_on: kafka. WebSocket endpoint: /ws.
frontend	5173	5173	Vite dev server. VITE_API_URL=http://localhost:8080.

9. Aderência aos Critérios de Avaliação (N1)

A tabela abaixo mapeia os critérios de avaliação da N1 para os artefatos entregues neste documento e nos demais repositórios do projeto.

Critério (N1)	Pontos	Evidência / Artefato
Coerência da Arquitetura	0,3	Diagrama C4 Contexto (Seção 2) e Contêineres (Seção 3). Justificativa Kafka vs REST (Seção 6). ADRs (Seção 7).
Qualidade da Interface	0,3	Protótipos Figma (entregue pela equipe de UI). Este documento especifica fluxos que guiam o design.
Documentação	0,2	Este documento C4 + Plano de Testes (documento separado). Diagramas claros com descrição de cada elemento.
Organização / Git	0,2	5 repositórios organizados na org GitHub do grupo. Hello World Kafka → WebSocket → Frontend já executado com sucesso.

Análise e Desenvolvimento de Sistemas | 2026/1
Documentação gerada para entrega N1 — Projeto Integrador